

PYTOOLBOX - CODE EXPLANATION & WORKFLOW

1. PROJECT

PyToolbox is a modular Python package.

Modules: cli.py, utils.py, eda_reporter.py, loaders.py, logistic_regression.py, scraper.py.

Tests live in tests/ and pyproject.toml stores package/dependency/test configuration.

2. MODULE DESIGN

Separation of concerns:

utils -> utilities

cli -> terminal interface

loaders -> CSV loading and preprocessing

eda_reporter -> exploratory data analysis

logistic_regression -> ML pipeline

scraper -> web scraping

3. INSTALLATION

```
python -m pip install -e ".[dev]"
```

. = current project, [dev] = development tools, -e = editable install.

4. CLI

CLI means Command Line Interface. We use argparse. Commands: clamp, palindrome, chunk.

Flow: terminal -> argparse -> parse_args() -> identify command -> call function -> print result.

build_parser() creates the parser. main() handles parsed arguments.

if __name__ == "__main__": main() runs main when cli.py is executed directly.

5. FUNCTIONS AND RETURN

Example:

```
def square(x):
```

```
    return x * x
```

```
result = square(5)
```

Flow: square(5) -> 5*5 -> return 25 -> result=25.

return sends a value back to the caller.

6. EDA

EDA means Exploratory Data Analysis.

EDAREporter stores a pandas DataFrame.

get_summary_stats(): df.describe().to_dict()

detect_missing_values(): df.isnull().sum()

calculate_correlation_matrix(): select numeric columns, then corr().

7. LOADERS

LoanDataLoader reads CSV data using pd.read_csv().

LoanPreprocessors creates engineered features:

```
Credit_Score_Sq = Credit_Score ** 2
```

```
DTI_Ratio_Sq = (DTI_Ratio ** 2).round(10)
```

The round handles floating-point values such as 0.16000000000000003.

8. LOGISTIC REGRESSION

LoanModelPipeline uses sklearn LogisticRegression by default.

Training: `model.fit(X_train, y_train)`

Prediction: `model.predict(X_test)`

Evaluation calculates accuracy, classification report and confusion matrix.

It returns a dictionary:

```
{"accuracy": acc, "report": report, "confusion_matrix": conf_matrices}
```

Then `result["accuracy"]` etc. can access named results.

9. SCRAPER

Flow: URL -> `requests.get()` -> HTML -> BeautifulSoup -> `self.soup` -> `extract_heading()`.

We fixed scraper/scrapers naming and installed BeautifulSoup4.

10. TESTING AND PYTEST

Testing checks expected behaviour. Example:

```
assert square(5) == 25
```

pytest is a Python testing framework.

`python -m pytest` runs pytest with the current Python interpreter.

`python -m pytest -v` gives verbose output.

Pytest discovers `test_*.py` files and `test_*` functions.

11. DEBUGGING

The first full test run found mismatches between older tests and current code:

- CLIApplication vs build_parser
- EDARreporter naming/method mismatch
- missing LoanDataLoader and LoanPreprocessors
- scraper.py vs scrapers.py
- SimpleWebScrapper vs SimpleWebScraper
- missing BeautifulSoup4
- empty Logistic Regression test file
- incorrect evaluate() return structure

We fixed each issue separately and reran focused tests before the full suite.

12. GIT/GITHUB WORKFLOW

local code -> `git status` -> `git pull --rebase origin main` -> resolve conflicts

-> run tests -> `git add` -> `git commit` -> `git push origin main` -> `git status`.

A commit is a local saved checkpoint. A push sends commits to GitHub.

We resolved a pyproject.toml rebase conflict using the VS Code Merge Editor, then

`git add pyproject.toml` and `git rebase --continue`.

13. FINAL STATE

`python -m pytest -v` -> complete test suite passed.

Changes were committed and pushed, and `git status` was clean/synchronized.

14. COMPLETE MENTAL MODEL

PYTHON -> functions/classes -> modules -> package -> dependencies -> CLI

-> testing -> debugging -> Git commit -> Git push -> GitHub

15. INTERVIEW ANSWER

I built a modular Python package called PyToolbox. I separated utilities, CLI, data loading, EDA, web scraping and a Logistic Regression pipeline into modules. I wrote pytest unit tests and used Git/GitHub for version control. During development I resolved interface mismatches, dependencies, floating-point precision and a rebase conflict.

KEY LESSON

Do not change code blindly. First identify whether the problem is in the implementation, test, dependency or project structure. Make a focused change and rerun the test.